```
# Name : Sai Rajendra Garudkar
# Year : Second Year
# Division : SA2
# Roll no. : SA228
# Subject : Data Analytics
```

```
# Problem Statement
# I am analyzing a weather dataset to understand how different environmental factors like temperature,
# humidity, precipitation, and wind speed vary across locations and time, and how they are related to each
# other. This analysis can help in identifying weather patterns and making better predictions for climate-related decisions


# The Dataset name is weather_data.csv, it is obtained from kaggle and contains
# multiple rows of weather observations with 6 columns: Location (city name), Date_Time (timestamp of observation), Tempera
# Humidity (percentage), Precipitation_mm (rainfall in mm), and Wind_Speed_kmh (wind speed in km/h).
```

```
# Importing Libraries
import pandas as pd
import seaborn as sns
import numpy as np
import matplotlib.pyplot as plt
import plotly.express as px
```

Dataset loading and display of 1st 5 rows and column with number of columns and rows

```
#Loading the dataset in notebook
df=pd.read_csv("weather_data.csv")
df
```

	Location	Date_Time	Temperature_C	Humidity_pct	Precipitation_mm	Wind_Speed_kmh
0	San Diego	2024-01-14 21:12:46	10.683001	41.195754	4.020119	8.233540
1	San Diego	2024-05-17 15:22:10	8.734140	58.319107	9.111623	27.715161
2	San Diego	2024-05-11 09:30:59	11.632436	38.820175	4.607511	28.732951
3	Philadelphia	2024-02-26 17:32:39	-8.628976	54.074474	3.183720	26.367303
4	San Antonio	2024-04-29 13:23:51	39.808213	72.899908	9.598282	29.898622
...	...	...	...	...	...	...
312231	Houston	2024-01-24 19:33:02	-7.118223	73.674194	0.645089	19.663296
312232	New York	2024-02-25 22:18:32	16.981712	76.338847	9.448736	18.131553
312233	Houston	2024-05-10 10:09:28	21.482723	60.503086	6.137766	13.399334
312234	Philadelphia	2024-01-12 10:05:19	4.334722	42.118165	6.165218	15.876594
312235	Chicago	2024-04-06 15:31:12	-3.027172	89.484890	NaN	NaN

312236 rows × 6 columns

```
#Display of 1st five rows
df.head()
#This helps us to understand the first five rows and makes us easier to analyze easily what the colums wants to express us.
```

	Location	Date_Time	Temperature_C	Humidity_pct	Precipitation_mm	Wind_Speed_kmh
0	San Diego	2024-01-14 21:12:46	10.683001	41.195754	4.020119	8.233540
1	San Diego	2024-05-17 15:22:10	8.734140	58.319107	9.111623	27.715161
2	San Diego	2024-05-11 09:30:59	11.632436	38.820175	4.607511	28.732951
3	Philadelphia	2024-02-26 17:32:39	-8.628976	54.074474	3.183720	26.367303
4	San Antonio	2024-04-29 13:23:51	39.808213	72.899908	9.598282	29.898622

```
df.shape
df.dtypes
#shape tells total rows & columns, and dtypes helps identify numeric vs categorical data.
```

	0
Location	object
Date_Time	object
Temperature_C	float64
Humidity_pct	float64
Precipitation_mm	float64
Wind_Speed_kmh	float64

dtype: object

Checking missing values

```
#Checking missing values
df.isnull().sum()
#This shows how many missing values are present in each column.
```

	0
Location	0
Date_Time	0
Temperature_C	0
Humidity_pct	0
Precipitation_mm	1
Wind_Speed_kmh	1

dtype: int64

```
# Handling missing values
# Temperature column
temp_mean = df['Temperature_C'].mean()
temp_median = df['Temperature_C'].median()
temp_mode = df['Temperature_C'].mode()[0]
temp_std = df['Temperature_C'].std()
temp_var = df['Temperature_C'].var()
temp_range = df['Temperature_C'].max() - df['Temperature_C'].min()
temp_midrange = (df['Temperature_C'].max() + df['Temperature_C'].min()) / 2

# Humidity column
hum_mean = df['Humidity_pct'].mean()
hum_median = df['Humidity_pct'].median()
hum_mode = df['Humidity_pct'].mode()[0]
hum_std = df['Humidity_pct'].std()
hum_var = df['Humidity_pct'].var()
hum_range = df['Humidity_pct'].max() - df['Humidity_pct'].min()
hum_midrange = (df['Humidity_pct'].max() + df['Humidity_pct'].min()) / 2

print("Temperature Stats:", temp_mean, temp_median, temp_mode, temp_std, temp_var, temp_range, temp_midrange)
print("Humidity Stats:", hum_mean, hum_median, hum_mode, hum_std, hum_var, hum_range, hum_midrange)
#These statistics help understand distribution and decide the best method to fill missing values.
```

```
Temperature Stats: 14.771234103860134 14.752424408794699 -19.95022342071841 14.488659513494248 209.92125449796737 59.9499154
Humidity Stats: 60.04252881340265 60.08410825735986 30.00022342039281 17.316387356771305 299.85727108974913 59.9997536480720
```

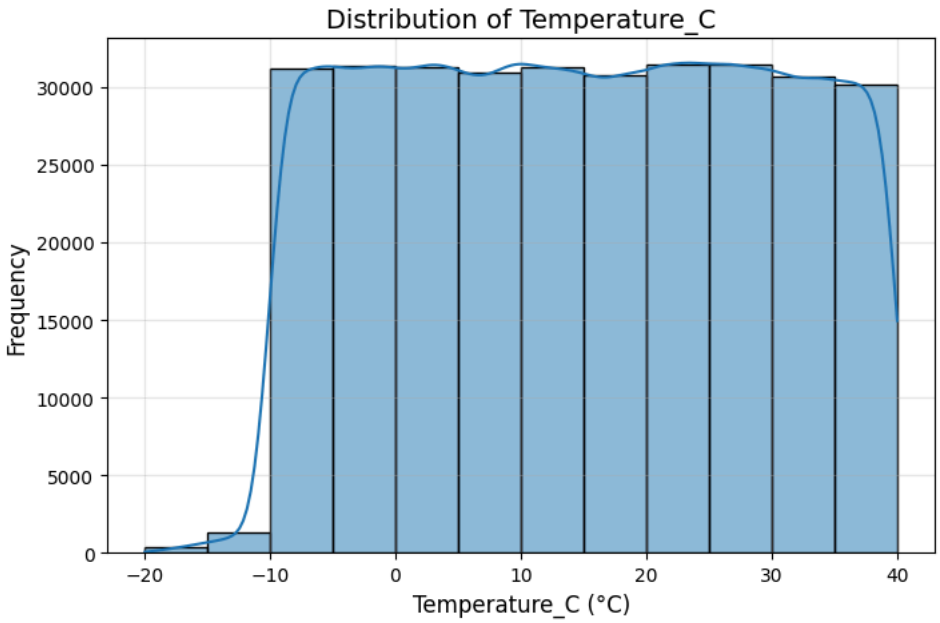
SUMMARY

```
#Summary of what issues found in the dataset



# 1. Missing values were found in several numeric columns such as Temperature, Humidity, Precipitation, and Wind Speed.
# 2. Statistical measures like mean, median, mode, standard deviation, variance, range, and midrange were calculated to unc
# 3. Missing values in numeric columns were filled using appropriate central tendency measures to maintain consistency and
# 4. Rows with missing values in important categorical fields like Location and Date_Time were removed to ensure data accur
# 5. Duplicate rows, if present, were also removed to prevent bias in the analysis.
```

Visualization

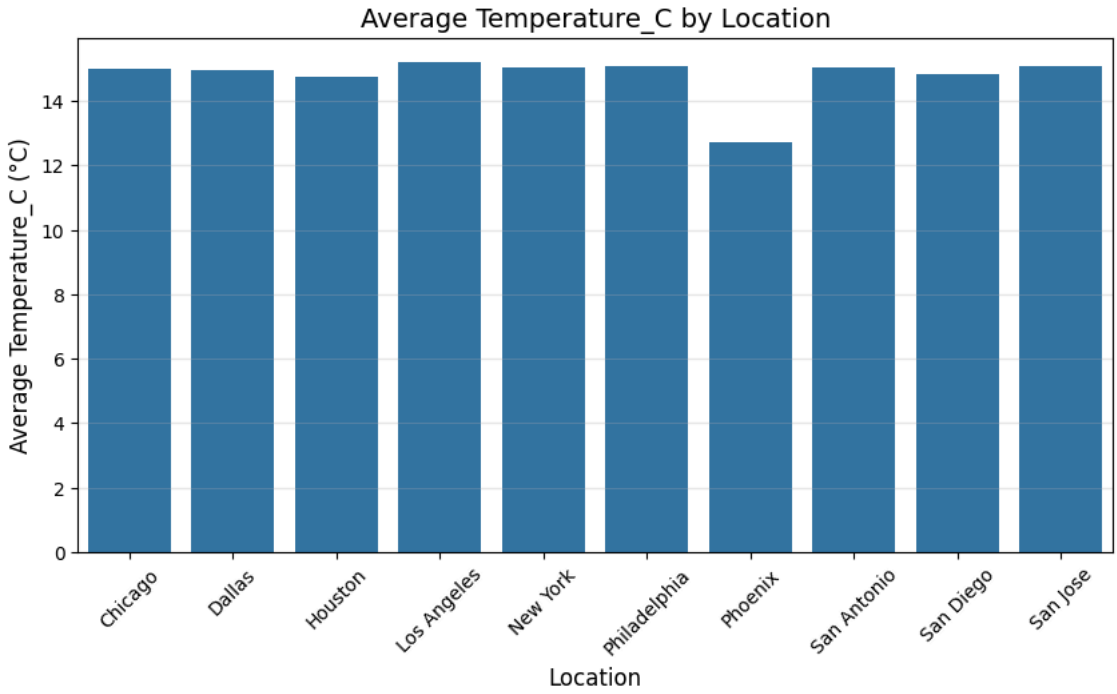
Histogram (Temperature Distribution)

```
# Histogram (Temperature Distribution)

plt.figure(figsize=(8,5))
sns.histplot(df['Temperature_C'], bins=12, kde=True)

plt.title("Distribution of Temperature_C", fontsize=14)
plt.xlabel("Temperature_C (°C)", fontsize=12)
plt.ylabel("Frequency", fontsize=12)

plt.grid(alpha=0.3)
plt.show()

#The histogram shows how temperature values are spread across the dataset.
#The curve helps identify whether the data follows a normal distribution or is skewed.
# The histogram shows that temperature values are mostly concentrated within a certain range, indicating the overall distrib
```



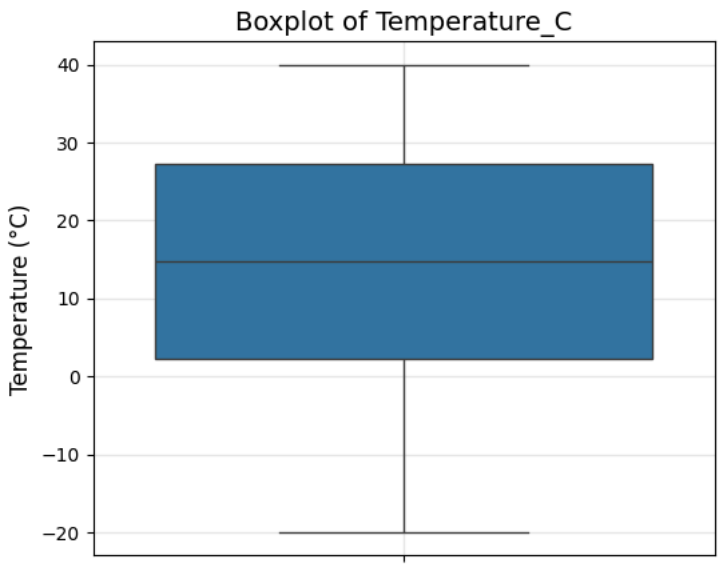
Bar Plot (Average Temperature by Location)

```
df.head()
plt.figure(figsize=(10,5))

avg_temp = df.groupby('Location')['Temperature_C'].mean().reset_index()

sns.barplot(x='Location', y='Temperature_C', data=avg_temp)

plt.title("Average Temperature_C by Location", fontsize=14)
plt.xlabel("Location", fontsize=12)
plt.ylabel("Average Temperature_C (°C)", fontsize=12)

plt.xticks(rotation=45)
plt.grid(axis='y', alpha=0.3)
plt.show()

#This chart compares average temperature across different locations.
#It helps identify which location experiences higher or lower temperatures.
# The bar plot shows variation in average temperature across locations, highlighting which areas are relatively hotter or co
```



Boxplot (Temperature Spread & Outliers)

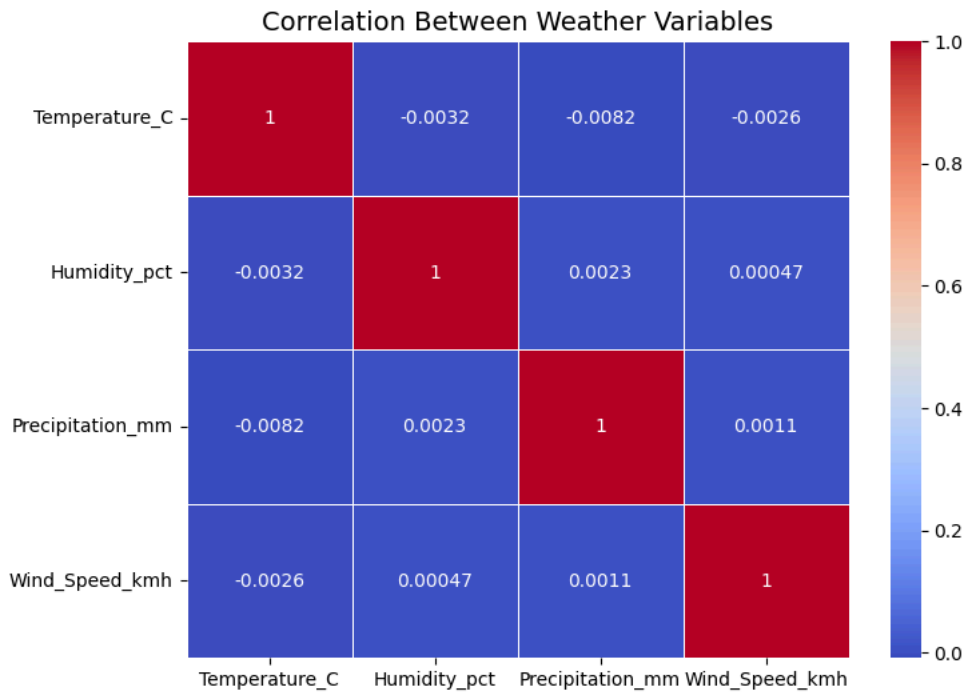
```
plt.figure(figsize=(6,5))

sns.boxplot(y=df['Temperature_C'])

plt.title("Boxplot of Temperature_C", fontsize=14)
plt.ylabel("Temperature (°C)", fontsize=12)

plt.grid(alpha=0.3)
plt.show()

#The boxplot shows the spread and central value of temperature data.
#Any points outside the whiskers indicate potential outliers.
#The boxplot indicates the spread of temperature data and reveals the presence of any outliers beyond the normal range.
```



Correlation Heatmap

```
plt.figure(figsize=(8,6))

corr = df.corr(numeric_only=True)

sns.heatmap(corr, annot=True, cmap='coolwarm', linewidths=0.5)

plt.title("Correlation Between Weather Variables", fontsize=14)
plt.show()
```

```
#The heatmap shows relationships between numeric variables like temperature, humidity, and wind speed.
#Values closer to 1 or -1 indicate strong correlation, while near 0 means weak relationship.
#The correlation heatmap shows the strength and direction of relationships between weather variables, helping identify posit
```



SIMPLE PREDICTION

Encoding Categorical Column

```
# Convert Location into numeric using encoding
df_encoded = pd.get_dummies(df, columns=['Location'], drop_first=True)
#Categorical data like location is converted into numeric form so the model can understand it.
```

Defining Features and Target

```
X = df_encoded.drop(['Temperature_C', 'Date_Time'], axis=1)
y = df_encoded['Temperature_C']
#X contains input features, and y is the target variable we want to predict.
```

Train-Test Split (80:20)

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
#Data is split so the model can learn on training data and be tested on unseen data.
```

Train Simple Model

```
from sklearn.linear_model import LinearRegression

# Impute missing values in X_train and X_test with the median
# This ensures that the model can be trained without NaN errors
X_train_imputed = X_train.fillna(X_train.median())
X_test_imputed = X_test.fillna(X_test.median())

model = LinearRegression()
model.fit(X_train_imputed, y_train)
#Linear Regression learns the relationship between input features and temperature.
```

```
▾ LinearRegression
LinearRegression()
```

Make Predictions

```
y_pred = model.predict(X_test)
#The model predicts temperature values based on test data inputs.
```

Check Accuracy

```
from sklearn.metrics import r2_score, mean_absolute_error

print("R2 Score:", r2_score(y_test, y_pred))
print("MAE:", mean_absolute_error(y_test, y_pred))
#R² shows how well the model fits the data, and MAE shows the average prediction error.
```

```
R2 Score: 0.0027681882334682495
MAE: 12.534272114355382
```

Final Markdown(What actually the result means)

1.The model was trained to predict temperature based on other weather conditions like humidity, wind speed, and location.

2.The results show how well the model can estimate temperature, with smaller errors indicating better predictions.

3.This helps in understanding how different weather factors influence temperature and can be useful for basic forecasting.

Findings From the Analysis

Finding 1: The histogram of temperature shows that most values are concentrated in a moderate range, indicating that extreme temperature conditions are less frequent in the dataset.

Finding 2: The bar plot of average temperature by location shows noticeable variation between locations, meaning some areas consistently experience higher temperatures than others.

Finding 3: The boxplot of temperature reveals the presence of a few outliers, indicating occasional extreme weather conditions that differ significantly from normal values.

Finding 4: The correlation heatmap shows a moderate relationship between some variables (such as humidity and precipitation), suggesting that certain weather factors are interdependent.

Real-World Recommendations

Recommendation 1 : Areas with higher average temperatures should focus on heat management strategies such as increasing green spaces and improving water availability to reduce the impact of heat.

Recommendation 2: Since some weather factors are related, monitoring key indicators like humidity can help in predicting rainfall and preparing better for weather changes.